octava

User manual

Table of contents

Table of Contents

Table of contents 2
Introduction 3
Installation 4
  Resource requirements 5
Usage 6
  Setting up a dataset 7
    Input files 7
    Additional options 9
  Editing a dataset 10
  Phenotype/sample view 11
  Association analysis 12
    Input filtering 12
    Association analysis options 13
    Models 13
    Association analysis results 14
  Gene/variant view 15
    Input options 15
    Results 16

Introduction

Octava is an application for the analysis of genetic variation and functional annotation. It is composed and designed as a pipeline:

1. The starting point are raw genotype and phenotype data;
2. The input can be visualized and processed by `filtering or transforming phenotype data` and by `filtering, imputing, phasing genotype data` , as well as constructing haplotype blocks;
3. Processed genotype and phenotype data can be used for `association analysis (GWAS)` using various models, with support for relatedness correction;
4. GWAS results in the form of `Manhattan plots can be visualized and compared` ;
5. Loci of interest can be `fine-mapped using additional statistical tools` ;
6. Loci or markers of interest can be studied on the functional side, analyzing and visualizing the `variant and gene annotation, impact on the protein level, gene expression` , gene enrichment, or gene localization and features.

However, **all (or most) parts of the application are independent and can be used separately**, as well as when some input files are not provided.

Octava runs as a local instance on a computer, with an **interface accessible through a web browser**. It is an **assembly of existing tools** and applications widely used in bioinformatics and genetic analysis, implemented in a single toolbox with an easy-to-use interface to the user. The main tools implemented are:

- `PLINK2` for genotype data processing and filtering;
- `PLINK1` for haplotype block construction;
- `BEAGLE` for genotype data imputation and phasing;
- `GEMMA`, `BLINK`, `GCTB` and `Random forest` for association analysis;
- `PiMASS`, `GCTB` and `Random forest` for statistical fine-mapping;
- `SnpEff` for variant annotation;
- `PROVEAN protein` for variant severity prediction;
- `IGV.js` for genome visualization;
- `Plotly.js` for plotting and visualization.

As input, it takes several files in formats standardly used in bioinformatics analysis, and supporting files in text format (see below). It supports the analysis of an arbitrary **diploid** genome, and can accommodate an arbitrary number of genomes at once.

Installation

Octava is provided as a Docker image (an isolated environment that contains all the requirements for running the app).

1. Create a directory on your system (later referred to as `data` directory). This will contain all your input files.
2. Install Docker desktop on your computer: follow the instructions for Windows, MacOS, Linux.
   - Requirements: Windows 10 or higher with WSL2 enabled; MacOS 11 (Big Sur) or newer; Linux 64-bit distribution with KVM virtualization
3. Navigate to the folder containing the Octava Docker image (`octava.tar.gz` or a similar name). No need to open or unzip the file.
4. Open command prompt in this folder:
   - On Windows, click the address bar (top of the file explorer), write `cmd` and press `↵Enter` .
   - On MacOS, double-click the folder and select `New Terminal at Folder` .
   - Or use the `cd` command in the command prompt to change directory.
5. Within the command prompt, write `docker image load -i octava.tar.gz` , press `↵Enter` to run (or replace `octava.tar.gz` with the name of the source file).
6. Installation will take several minutes and require up to 10GB of free storage.
7. After installation, open the Docker desktop application.
8. Go to the `Images` tab, select the `octava` image and click `▶(Run)` .
9. In the dialog window, expand `Optional settings` , enter any name. Under `Ports` , enter 5101 (unless you have a reason to use a different port).
10. Under `Volumes` , click `…` and navigate to the `data` directory from step 1. Open the folder and click `Select folder` . Under `Container path` , enter `/octava/data/` .
11. Click `▶(Run)` .
12. You will access your application in your web browser, at `localhost:5101` (or your custom port).
13. You can stop the application under `Containers` tab on the right, by clicking `■(Stop)` .
14. To start Octava again, go to `Containers` and start it using `▶(Run)` , without repeating any of the steps above.
15. The application will **update to the latest version every time** the docker container is restarted.

Resource requirements

Storage

Octava supports an arbitrary number of datasets concurrently set up for analysis. Apart from the input files provided by the user, additional files will be created. Make sure you have sufficient storage for each dataset added. A rule of thumb is to allocate storage two to three times as much as all your input files combined.

Processing power

Processing power should not be a limiting factor for analysis. With less powerful processor or fewer CPU cores, analysis may take longer, but should be feasible on standard personal computers.

Memory

Memory usage by Octava is dependent on memory usage by the background tools that conduct analysis (PLINK, BEAGLE). While gene/variant view should not be resource-intensive, association analysis and especially genotype data filtering/processing may require a lot of memory. For genotype input hundreds of gigabytes in size, successful processing may require tens to hundreds of gigabytes of RAM, and you may encounter “Out of memory” errors. In that case, process your input data using the standalone scripts (contact peter.polacek@wur.nl) in a high performance computing environment and copy the output files to your local machine for further analysis.

Usage

In Octava, data is organized into datasets. Each dataset must contain at least a variant file, together with the corresponding genome sequence file that was used for variant calling, and a genome annotation file matching the genome sequence file. Before a dataset can be visualized and analyzed in Octava, it needs to be set up first.

Setting up a dataset

> Depending on the size of your input data, this may take on the order of hours (or up to days for terabyte-size data), but is a one-time requirement.

Each dataset is organized as a folder in the `data` directory. The name of the folder is used as the name of the dataset; the **name can be any valid name, and can include spaces, commas, periods**, etc. Place all your input files (see below) in this folder. This directory will be recognized when you go to `Add new dataset` .



With an input folder organized and located in the `data` directory, set up the dataset for use in the app by choosing your folder on the `Add new dataset` page. For each input file, specify the corresponding file and `Submit` .

Input files

1. **Icon**
   - Optional, `PNG` or `JPEG` format
   - Only visual, identifies a dataset on the home page
   - Square aspect ratio, will be scaled to 128x128px
   - Maximum size: 1MB
2. **Variant file**
   - `VCF` or `BCF` format
   - Contains input genotype data
   - Uncompressed, or compressed using `bgzip` (not gzip)
   - Can be indexed beforehand using `tabix`

     > Certain chromosome names will cause problems. **DO NOT** use the format `chr1,chr2,…` or similar. Use numbers (`1,2,…`) or **preferably**, word-like names (`Species.v1.0.chr1`). Replace chromosome names using `bcftools annotate --rename-chrs` , or use `awk` or `sed` commands.

3. **Annotation file**
   - `GTF` , `GFF` or `GFF3` format
   - Annotation of genes and gene features (commonly available for most genome assemblies)
   - Uncompressed, or compressed using `bgzip` (not gzip)
   - Can be indexed beforehand using `tabix`

     > Functional annotations of genes in the annotation file (function, keywords, GO terms etc.) will not be recognized and displayed in the software output. To retain these, extract them and provide them via the `gene info file` (see below).

   -
4. **Sequence file**
   - `FASTA` format
   - DNA sequences of all chromosomes, contigs or scaffolds (available for all genome assemblies)
   - Uncompressed, or compressed using `bgzip` (not gzip)
   - Can be indexed beforehand using `samtools faidx` (both `.fai` and `.gzi` files are required)

> Names and lengths of chromosomes, scaffolds or contigs **must match** between the sequence file and the variant file. For mismatched names, rename them in one file or the other. For mismatched lengths, use the correct sequence file.

5. **Phenotype file**
   - Optional, tab-delimited text file (`TXT` or `TSV` format)
   - Phenotype data for input samples, must be quantitative or binary (categorical phenotype values are not supported)
   - First row must contain trait names, first column must contain sample identifiers (not all samples have to be present)
   - The first cell (intersection of first row and first column) should have a column label as well, such as “Sample” or “Accession”

   > In trait names, characters other than letters, numbers or underscores will be replaced with an underscore.

6. **Gene expression file**
   - Optional, tab-delimited text file (`TXT` or `TSV` format)
   - Contains gene expression data, normalized if necessary (for reliable comparison between runs)
   - First row must contain study run identifiers, first column must contain gene identifiers (not all genes have to be present)
   - The first cell (intersection of first row and first column) should have a column label as well, such as “Gene”

7. **Run info file**
   - Optional, tab-delimited text file (`TXT` or `TSV` format)
   - Optional, contains metadata about gene expression runs
   - First row must contain metadata column names, first column must contain run identifiers, second column must contain study identifiers

8. **Study info file**
   - Optional, tab-delimited text file (`TXT` or `TSV` format)
   - Optional, contains metadata about gene expression studies
   - First row must contain metadata column names, first column must contain study identifiers

   > Study identifiers must match between study info file (first column) and run info file (second column).

9. **Sample info file**
   - Optional, tab-delimited text file (`TXT` or `TSV` format)
   - Optional, contains sample metadata
   - First row must contain metadata column names, first column must contain sample identifiers (not all samples have to be present), second column must contain sample names (can be identical to sample identifiers)
   - Sample names are used instead of sample identifiers in the software, to allow for more human-friendly names

10. **Gene info file**
    - Optional, tab-delimited text file (`TXT` or `TSV` format)

- Contains gene metadata (functional annotation, gene ontology, additional identifiers, …)
- First row must contain metadata column names, first column must contain gene identifiers (not all genes have to be present)

11. **BAM files**
    - Optional, BAM format
    - Can be indexed beforehand using `samtools index`

Sample IDs **must match** between `variant file`, `phenotype file` and `sample info file`.

Gene IDs **must match** between `annotation file`, `gene expression file` and `gene info file`.

Files can be provided in the dataset directory as links (hard or symbolic).

If an uncompressed variant, annotation, sequence or gene expression file is given as a symbolic link, the link will be removed and the file will be hard-copied to the dataset directory and compressed.

If a phenotype or sample information file is given as a symbolic link, the link will be removed and the file will be hard-copied to the dataset directory.

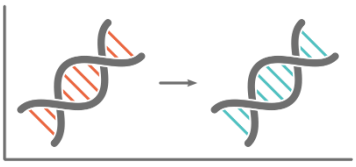
Additional options

1. **Enrichment**
    - Requires `gene info file`
    - Gene enrichment can be calculated for chosen columns, for each unique value in a column
2. **Concatenate contigs**
    - For genome annotations with a large number of chromosomes (e.g. the presence of thousands of unplaced contigs), visualisation will be unappealing, slow, or fail altogether. Select these contigs under this option, they will be grouped under the label “Other”. This does not affect analysis, only visualisation.

After setting up a new dataset, it can be accessed on the home page, by clicking the corresponding icon and label.

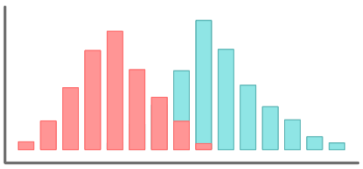
Editing a dataset

For now, support is only added for uploading additional trait data. The requirements for a file containing additional phenotypes are the same as the input `phenotype file` (see above). Maximum file size is 5MB. New traits will be merged with the existing phenotype data by appending to the end.



Phenotype/sample view

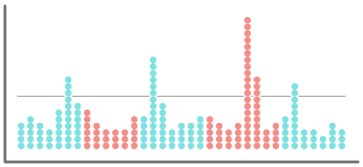
Browse and filter samples by metadata, compare traits and visualize sample distribution, process phenotype data and group traits or samples into groups.



1. **Sample filtering**
   - View sample metadata, provided via the `Sample info file`
   - View per-sample `Missingness` and `Heterozygosity` : the proportion of variant positions called as missing (out of total), and the proportion of variant positions called as heterozygous (out of all non-missing), respectively (calculated on the input variant data)
   - Samples table can be filtered to include only samples phenotyped for a given `Trait` using
2. **Trait filtering**
   - Apply a `Minimum` or `Maximum` threshold to filter trait values
3. **Compare traits**
   - Select one or more traits or trait groups, see trait distribution and correlation
   - **Double-click a plot** to zoom into a single histogram plot or a pair of trait histograms and their correlation (and double-click to zoom out)
   - Divide plotted samples by selected sample groups
4. **Transform traits**
   - *Log(e) transform* – values of a trait with all positive values will be converted to their log(e) value
   - *Box-cox transform* – calculates the optimal power parameter to transform values based on the box-cox transform function
   - *PCA transform* – principal components of a group of traits will be calculated
   - *Z-score normalization* – values will be normalized to a scale with mean = 0 and standard deviation of 1
   - *Make binary* – values below `Threshold` will become 0, values at or above will become 1

Association analysis

View genotype data statistics, apply filters to create curated marker sets, construct haplotype blocks. Carry out association analysis using various statistical models, visualize genome-wide significance and compare results, fine-map loci of interest using additional statistical tools.



Input filtering

5. **Plot**
   - *Marker density* – number of markers in each 1Mbp window across the genome
   - *Linkage disequilibrium* – per-marker LD by chromosome, on a subset of markers uniformly distributed across the chromosome
   - Remaining plots – histograms of each metric, showing sample or variant distribution
6. **Apply to existing**
   - Filtering and processing will be applied to an existing `Marker set` , instead of the raw VCF
   - Can be used **only on SNP sets**, not haplotype sets

   To apply filters multiple times (and check filtering quality via histograms), apply filters once, **reapply to the intermediate set**, and delete the intermediate set when finished.

   The existing marker set used with this option will be the set with **filters and imputation applied, but not LD pruning.** This might be changed in the future.
7. **Quality filters**
   - Filtering applied using PLINK2
   - Applied in a single step; the order is controlled by PLINK2 (sample groups → heterozygosity → missingness → MAF → HWE)

   To apply filters in a specific order, apply first filter to an intermediate set, apply second filter **to existing intermediate set** and delete the intermediate set when finished.
8. **Imputation**
   - Fills in missing genotype data and phases genotypes, using BEAGLE default settings
   - Applied **after quality filters**
9. **Linkage disequilibrium pruning**
   - Applied **after imputation**, using PLINK2 `--indep-pairphase <window> <threshold>`
10. **Haplotype construction**
   - *LD-based* – using PLINK1.9 `--blocks --blocks-max-kb <max size>`
   - *SNP number window, Base-pair window* – using a custom script

   SNP marker data will be genotyped based on haplotype blocks by **treating each haplotype in a block as a marker**, generating allele dosage data for each. Beforehand, haplotypes are **filtered with a MAF threshold of 0.05 and unplaced markers are removed**.

Association analysis options

1. **Plot**
   - *Marker density* – number of markers in each 1Mbp window across the genome
   - *Block distribution* – only for haplotype marker sets; size and position of each haplotype block across the chromosomes
   - *Linkage disequilibrium* – per-marker LD by chromosome, on a subset of markers uniformly distributed across the chromosome
   - *Kinship matrix* – heatmap of the Hamming distance between samples based on genotype data
   - *Principal components* – 3D scatter plot of first three principal components of the genotype data, with the percentages of variance explained of the first ten principal components
   - Remaining plots – histograms of each metric, showing sample or variant distribution

     Decimal numbers cannot be accurately represented in binary form and used by a computer. For example, the number 0.1 might be represented as 0.0999... In case you apply a *minor allele frequency* threshold of 0.1 (or in any similar filtering case), variants with MAF at exactly 0.1 might be treated as 0.09999… and plotted as being below the minimum threshold. This is **not an error in the filtering** – no variants with a value truly below the threshold were retained – it is instead an artifact only affecting the histogram plot.
2. **Parameters**
   - Specify the desired `Marker set`, `Model` and `Traits` to run the analysis
   - Define your population structure correction (some models have requirements to apply/not apply correction)
   - Kinship and PC correction is applied using kinship matrix and principal components calculated from the specified Marker set by default, change this via `K/Q from`
   - Provide a file with `Additional covariates` (see ? for more information)

Models

1. **GLM**
   - Generalized linear model via GEMMA, using `-lm 1` (1 sets the Wald test as the statistical test)
   - Kinship correction is not supported, PC correction is optional
2. **MLM**
   - Mixed linear model via GEMMA, using `-lmm 1` (1 sets the Wald test as the statistical test)
   - Kinship correction is required, PC correction is optional
3. **Multi-trait MLM**
   - Identical to MLM (above), but runs with multiple traits at the same time
   - The analysis reports p-value significance for the combined trait analysis
   - Traits must be saved as a trait group to run multi-trait MLM.
4. **BLINK**
   - BLINK model
5. **BayesB**
   - Assumes most markers have no effect on the trait (more about BayesB and BayesC in GCTB manual)
6. **BayesC**
   - Assumes many markers have an effect on the trait

7. **Random forest**
   - Uses the python package `RangerForestRegressor`
   - Grows a random forest a number of times specified by `Iterations` , with the *mtry* parameter specified using `Test markers` and *ntree* parameter specified using `Trees per forest`
   - Uses **permutation importance** as the importance metric, with the final importance per marker calculated as the average importance across iterations
   - Corrects for false positives by shuffling phenotype values and growing a random forest a number of times specified by `Permutations` , subtracting the 99th percentile of the importance values of shuffled phenotype data from the average real importance

     Parameter optimization:
     mtry and *ntree* parameters can be optimized. Enter values to test in `Optimize parameters` separated by a comma and a space (,). The result shows proportion of variance explained (PVE) per *mtry*/ntree combination, plotted with *ntree* as the x-axis and *mtry* as individual lines. Look for the lowest *ntree* value where the lines level out, and the lowest *mtry* value for which increasing *mtry* does not result in higher PVE. For these optimal *mtry/ntree* values, setting a higher value would not result in higher quality model, only a slower speed.

Association analysis results

1. **View interactive Manhattan plot(s)**
   - Specify the desired `Marker sets` , `Models` and `Traits` to plot
   - Specify a P-value threshold to plot

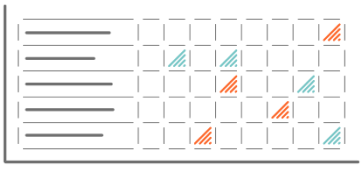
     The default threshold plotted is fixed to 8. Corrected threshold is not calculated, because results obtained through different methods will have different corrected thresholds.
     To calculate a Bonferroni-corrected threshold, i.e. $-\log_{10}(0.05/number\ of\ markers)$, find the number of markers by right-clicking your chosen Marker set.
   - Plot multiple Manhattan plots as a grouped plot for an alternative way of comparing results

     Plot as groups:
     Plotted data will be grouped by `Window` , with a single marker within each window. Marker size represents the number of runs with significant association within the window (at or above `Threshold`). To plot multiple groups, each must be saved as a Trait group. See ? for more information.
   - Interactive plot only shows top 5,000 significant markers; **view full plot and QQ-plot** (as static plots) by clicking `Switch plots`
2. **Loci of interest**
   - Zoom into a locus within a single chromosome to ⊕ save the locus, `View LD` of the locus, or analyze it in `Gene/variant view`
3. **Fine-mapping**
   - Within one or multiple loci of interest, re-analyze marker association using Bayesian models (BayesB, BayeC, PiMASS) or using random forest
   - Analyzes loci with the full marker set, rather than the LD-pruned set

     Fine-mapping results are lost after closing the page, and must be ⊕ saved to be retained.

Gene/variant view

Specify loci or genes of interest and view variant annotation, genotype data, SnpEff annotation, PROVEAN severity prediction and variant correlation with phenotype; view gene information, annotation, expression, functional enrichment; view position of genes within the genome and of variants within a gene.



Input options

Many features of the **Gene/variant view** branch depend on **SnpEff annotation** of the input VCF file. Make sure to annotate your VCF file beforehand or while **Setting up a dataset** for best results.

1. **Markers**
   - View variant data in `Raw VCF` (the unfiltered input VCF file), or in a filtered marker set created via the `Association analysis` branch

     Genotype data (call quality, allelic depths, etc.) and SnpEff annotations **will be lost** for all variants in a filtered marker set, and PROVEAN prediction will not be possible.
     Use this option (specifying a filtered set instead of Raw VCF) e.g. for viewing genotype call data per haplotype, or analyzing phenotype correlation with a haplotype.

2. **Target**
   - Limit variant search to only coding regions or only promoter regions; can greatly speed up loading time
   - Promoter regions defined as *n* base pairs upstream of transcript start (respecting the gene strand)

     Analyzing a locus of interest identified via Association analysis may better be done on the **Entire region**, rather than only **Coding regions**. Remember to switch this option if desired.

3. **Variant impact**
   - Filter variants by the SnpEff-annotated `Annotation_Impact` attribute
   - Select `Don't show variants` if you are only interested in the genes (greatly speeds up loading time)

4. **Filter by**
   - `Sample group`: filter variants by sample (requires a sample group to be created through **Phenotype/sample view**)
   - `Type` : filter variants by type (based on variant length or existing annotation generated by variant caller)
   - `Consequence` : filter variants by SnpEff-annotated `Annotation` attribute (Sequence Ontology nomenclature)

5. **Region(s) of interest**
   - `Region`, given the chromosome, start and end
   - `Gene ID`, or a list of IDs separated by a comma and space (,)
   - `Coordinates`, in the format **Chr:start-end**, or a list of coordinates separated by a comma and space (,)

6. **Variant quality**
   - `Missingness` : proportion of missing genotypes out of total
   - `Heterozygosity` : proportion of genotypes out of all non-missing
   - `MAF` : frequency of the minor allele out of all called alleles
   - Calculated **after filtering by sample**
7. **Skip**
   - Hide samples or variants that do not contain meaningful variation (see ? for more information)

Results

1. **Variants**
   - Table of variant information and genotype calls
   - `Affected gene` – all the SnpEff-annotated `Gene_ID` attributes of the relevant annotations
   - `Missingness`, `Heterozygosity`, `Minor allele frequency` – calculated on the subset of genotypes **filtered for sample group selection**
   - `Annotation` – selected variant information: type + ordinal | sequence | *Annotation_Impact* | *Annotation* | *HGVS.p* | *ERRORS / WARNINGS / INFO* (attributes in italic are taken from SnpEff annotation)
   - `Severity score` – calculated for selected variants using the `Annotate severity` button and retained permanently once calculated, see below for more explanation
   - *Genotype calls* – missing allele denoted as `–`, reference allele denoted as `·`, alternative allele denoted as `type + ordinal`. Phasing (`|` or `/`) is retained in the call representation
   - Sample labels used in the table are the human-friendly **Name** (second column of the Sample info file), not the VCF-friendly **Identifier** (first column of the file)
   - See ? for explanation of **color-coding schemes**

     Genotype quality (for color-coding):
     If variant call has ***GQ*** attribute – quality score is the attribute value
     If variant call has ***PL*** attribute – quality score is the second lowest attribute value (lowest is typically zero)
     If variant call has ***GL*** attribute – quality score is the second lowest attribute value to the power of -10 (genotype likelihood converted to phred-scaled genotype likelihood)
     Quality score value is set to a maximum of 55 and mapped to a red-yellow-green color scale

     Right-click functionality:
     Position – all attributes at a variant position, annotated by the variant caller
     Reference – show full sequence
     Annotation – all SnpEff-annotated attributes (see SnpEff manual for more explanation)
     Severity score – all attributes for PROVEAN prediction (see below for more explanation)
     Genotype call – all attributes for the call, annotated by the variant caller

     If a subset of genes (rows) is selected in the **Genes** table, you can **Copy selection**, i.e. unselect all variants located in genes not in the Genes table selection. This **only unselects** variants, unselected variants located in genes that are part of the Genes table selection will remain unselected.

PROVEAN severity prediction:
Calculated using the PROVEAN algorithm methodology, implemented using a custom script. The algorithm:

- Uses BLAST to find homologs of the protein of interest, with an E-value above 0.1
- Clusters homologs by similarity, with an identity threshold of 0.8
- Based on average cluster E-value (similarity, takes up to 45 most similar clusters
- Reference and mutated sequences are aligned to each homolog within cluster using NEEDLE, with a delta score for the alignment calculated using the BLOSUM62 matrix
- Average delta per cluster is calculated, and the average of cluster averages is the final severity score
- Severity score threshold of +/- 2.5 is used to assign the prediction of DELETERIOUS REF/ALT, respectively

The full prediction information (**right-click**) contains: variant type + ordinal, *Feature_ID*, *Gene_ID*, *Gene_Name*, *Feature_Type*, *Annotation*, *HGVS.p*, final mean severity score, standard deviation of the final mean, number of homologs used, number of clusters used, severity prediction (attributes in italic are taken from SnpEff annotation). See PROVEAN documentation for more information about the algorithm.

2. **Variant-phenotype correlation**
   - Boxplot of unique genotype values against the respective phenotype values by sample, for a chosen trait
   - **Left-click** a variant within the plot to **(un)select the variant** in the Variants table (select or unselect all variants using the `Select all` button of the Variants section to toggle between selecting variants in or out)

   To find meaningful correlation between a variant and a trait, filter variants by minor allele frequency beforehand.

3. **Genes**
   - Table of basic gene information (position, strand), number of variants within (only the variants filtered based on search query), and additional annotations provided via the Gene info file

   If a subset of variants (columns) is selected in the **Variants** table, you can **Copy selection**, i.e. filter out all genes with no selected variants within. This **only filters out** genes, filtered out genes with selected variants within will remain filtered out.

4. **Gene expression**
   - Boxplots of gene expression by attribute or group, for genes selected through the Genes table
   - Based on data provided via the Gene expression file, Run info file and Study info file
   - Gene expression study runs can be filtered by `Study` or by `Run`
   - Runs can be grouped and expression data plotted by groups
   - **Left-click** a gene within the plot to **(un)select the gene** in the Genes table (select or unselect all genes using the `ID` column dropdown to toggle between selecting variants in or out)

5. **Gene enrichment**
   - Given the set of genes selected via the Genes table and a chosen annotation attribute, calculates for each unique attribute value the $\log_2$-fold enrichment and the $\log_{10}$-p value of the probability of finding as many or more occurrences of the value in the set (using hypergeometric distribution)

6. **Gene view**
   - Visualisation of the genome or specified regions of interest using `IGV.js`
   - Limit browser window to genes selected in the Genes table using
   - Selected genes highlighted in red spans, selected variants highlighted in blue spans
   - Variant information is limited to position information (genotype calls are not shown) for performance reasons
   - For more information about the functionality, see IGV.js manual
7. **Sample clusters**
   - Table of samples clustered by identity across the queried variants
   - **Static**, unselecting variants through the Variants table does not affect the Sample clusters table
8. **Edit query**
   - Functionally identical to the Input page, allows editing input parameters in-place